Michael Flanagan DSA CA1

x24269654

GitHub Repository Link: https://github.com/MichaelF221/CA1-DSA-Bike-Tracker.git

Theme: Mobility Innovation

The Docklands DublinBikes Tracker is an application that lets people in the Docklands area log and track problems/issues with the DublinBikes stations, like stations being empty, being full/crowded, or docks/terminals not working. These issues can cause disruptions to everyday commuters and their journeys as some users may arrive at the station not being able to use or return a bike. This app helps by storing reports using ADT's, so the reports can be handled in order, with urgent ones being dealt with first. This supports the Mobility Innovation theme because it helps make bike journeys more reliable by showing what stations have constant issues and which need to be dealt with first.

Core Functionality

The app is based around a BikeReport class, which contains:

- reportId (int)
- stationName (String)
- issueType (String) (Empty/Full/BrokenDock/Terminal/Other)
- details (String)
- severity (int) (1-10)
- resolved (Boolean)

When a report is created, it will show as resolved = false. It can later be marked as resolved by the user after processing.

Inheritance

There will be two report types, such as:

- BikeReport (parent class)
- AvailabilityReport extends BikeReport (Empty/Full)
- FaultReport extends BikeReport (BrokenDock/Terminal)

Reports will be stored as BikeReport so the same ADT code can work on both types, while still allowing two different report categories.

Data Storage (Queue using ArrayList)

All reports are stored in the Queue called reportQueue using the ArrayList-based (MyQueue implements QueueInterface). The methods used are:

- enqueue(Object e) add report
- dequeue() remove oldest report
- frontofQueue() view oldest report (not being removed)
- isEmpty(), size()

CRUD on Queue

- Create: enqueue(report)
- Read: display all reports currently stored in the queue
- Update: rebuilds queue by dequeuing each item and re-enqueuing it after updating the matching reportId
- Delete: rebuild queue the same way, skipping the matching reportId

A singly linked list is included using SLList and Node, which will be used to store and display the saved reports.

Urgent List (Priority Queue using ArrayList)

Urgent reports are stored in a Priority Queue called urgentQueue using the ArrayList priority queue (MyPriorityQueue implements PQInterface with PQElement). The priority key is severity, using the methods as follows:

- enqueue(int priorityKey, Object element)
- dequeue() (returns most urgent)
- size(), isEmpty(), printPQueue()

CRUD on Priority Queue

- Create: enqueue(severity, report)
- Read: display/print the queue
- Update: remove the report then re-enqueue() with updated details
- Delete: remove the report from the list

User Action History (Stack)

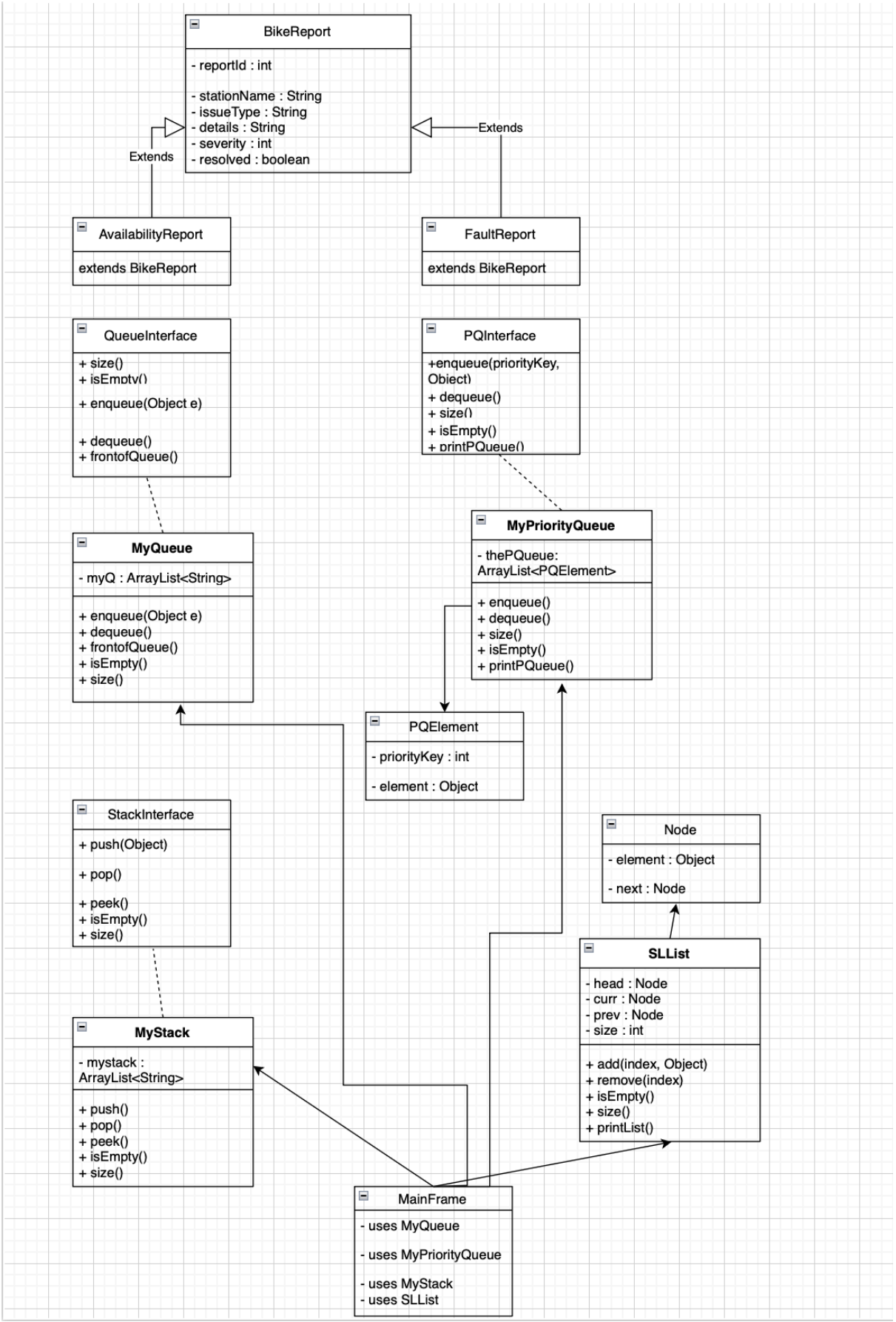
A stack called actionStack stores messages like “Added report 3”. It uses the Stack (MyStack implements StackInterface) using an ArrayList, like so:

- push(Object e)
- pop()
- peek()
- isEmpty()
- size()

GUI

The GUI will have basic input fields and buttons like: Add, Display All, Update, Delete, Next, Dequeue Most Urgent, and Recent Action. The app will run from the main class and show some testing outputs for Queue, Stack, and Priority Queue operations.

<u>Class Diagram</u>


BikeReport
- reportId : int
- stationName : String
- issueType : String
- details : String
- severity : int
- resolved : boolean
Extends
Extends
AvailabilityReport
extends BikeReport
FaultReport
extends BikeReport
QueueInterface
+ size()
+ isEmpty()
+ enqueue(Object e)
+ dequeue()
+ frontofQueue()
PQInterface
+enqueue(priorityKey, Object)
+ dequeue()
+ size()
+ isEmpty()
+ printPQueue()
MyQueue
- myQ : ArrayList<String>
+ enqueue(Object e)
+ dequeue()
+ frontofQueue()
+ isEmpty()
+ size()
MyPriorityQueue
- thePQueue: ArrayList<PQElement>
+ enqueue()
+ dequeue()
+ size()
+ isEmpty()
+ printPQueue()
PQElement
- priorityKey : int
- element : Object
StackInterface
+ push(Object)
+ pop()
+ peek()
+ isEmpty()
+ size()
Node
- element : Object
- next : Node
SLList
- head : Node
- curr : Node
- prev : Node
- size : int
+ add(index, Object)
+ remove(index)
+ isEmpty()
+ size()
+ printList()
MyStack
- mystack : ArrayList<String>
+ push()
+ pop()
+ peek()
+ isEmpty()
+ size()
MainFrame
- uses MyQueue
- uses MyPriorityQueue
- uses MyStack
- uses SLList